

NLP 2026 - Text Classification & Sentiment Analysis

Lecture 2 | From raw text to structured predictions
Wednesday, 29 April 2026 | 08:00-12:00

Prof. Dr. Paul Pronobis
Free University of Bozen-Bolzano

Lecture 2 Roadmap

Text Classification & Sentiment Analysis

Today at a glance

Four parts

1. What is text classification?
2. Sentiment analysis deep dive
3. Evaluation metrics
4. Classical ML vs. transformers

Learning objectives

- Distinguish binary, multi-class, and multi-label tasks
- Build sentiment analysis pipelines
- Interpret classification metrics
- Compare classical and transformer approaches

Why this lecture matters

Text classification turns messy language into decisions: spam or not, complaint or praise, urgent or routine, positive or negative.

Business value

- Scale customer feedback analysis
- Automate routing and prioritization
- Monitor brands and markets in real time

Technical value

- Core supervised learning problem in NLP
- Foundation for many downstream applications
- Great lens for understanding model trade-offs

Quick Poll

Have you ever left a review online?



Yes

Product, restaurant, hotel, app,
or service



No

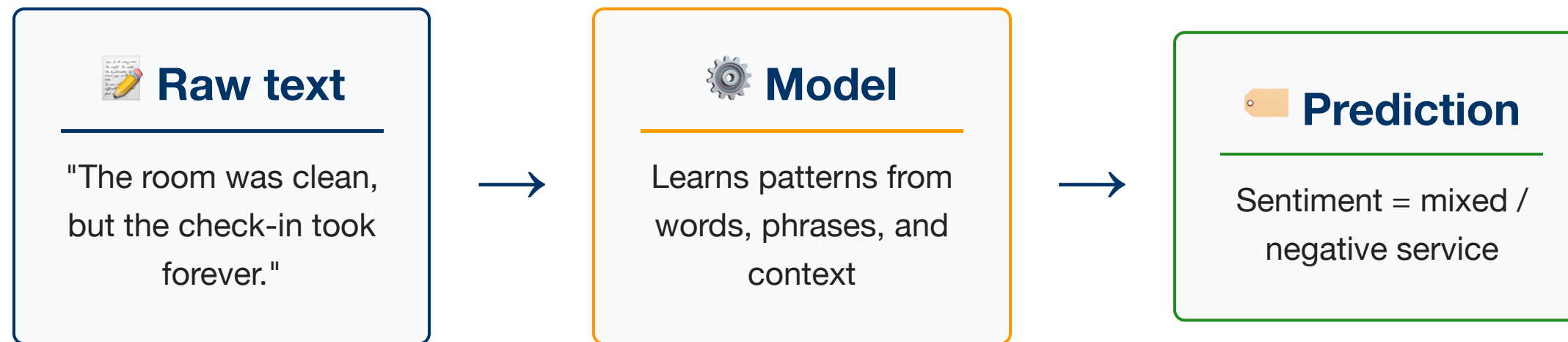
But you have probably read
many

Even if you never write reviews, your decisions are still influenced by classified opinions.

Part 1

What is Text Classification?

From unstructured text to structured predictions



Binary vs. multi-class vs. multi-label

Type	Question	Example labels	Typical use case
Binary	One of two classes?	spam / not spam	Email filtering
Multi-class	Exactly one out of many?	sports, politics, finance	Topic labeling
Multi-label	Several labels at once?	urgent, complaint, billing	Support ticket routing

Key distinction: multi-class chooses one label, multi-label can assign many.

Three task formats, one intuition

1 Binary

Is this email spam?

Output: yes or no

2 Multi-class

Which department should answer?

Output: sales or support or legal

3 Multi-label

Which tags apply?

Output: urgent + billing + negative

Real-world applications

Spam detection

Protect inboxes from irrelevant or malicious content.

Topic labeling

Organize news articles, documents, and support requests.

Intent recognition

Understand what a user wants in chatbots or voice assistants.

Sentiment analysis

Summarize opinions in reviews, surveys, and social media.

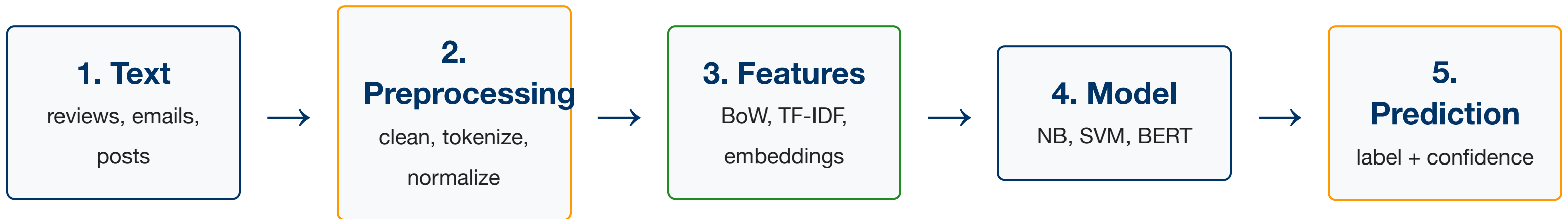
Risk monitoring

Flag compliance issues, fraud signals, or harmful content.

Recommendation metadata

Assign categories and tags that power search and discovery.

The text classification pipeline



Classical features: Bag of Words and TF-IDF

Method	Idea	Strength	Weakness
Bag of Words	Count how often terms appear, ignore order.	Fast and simple	No context
TF-IDF	Upweight informative terms, downweight common ones.	Better signal quality	Still mostly bag-of-words

TF-IDF intuition with three documents

Documents

- **D1:** hotel room clean clean breakfast good
- **D2:** hotel location good breakfast
- **D3:** flight delay delay delay bad

Intuition

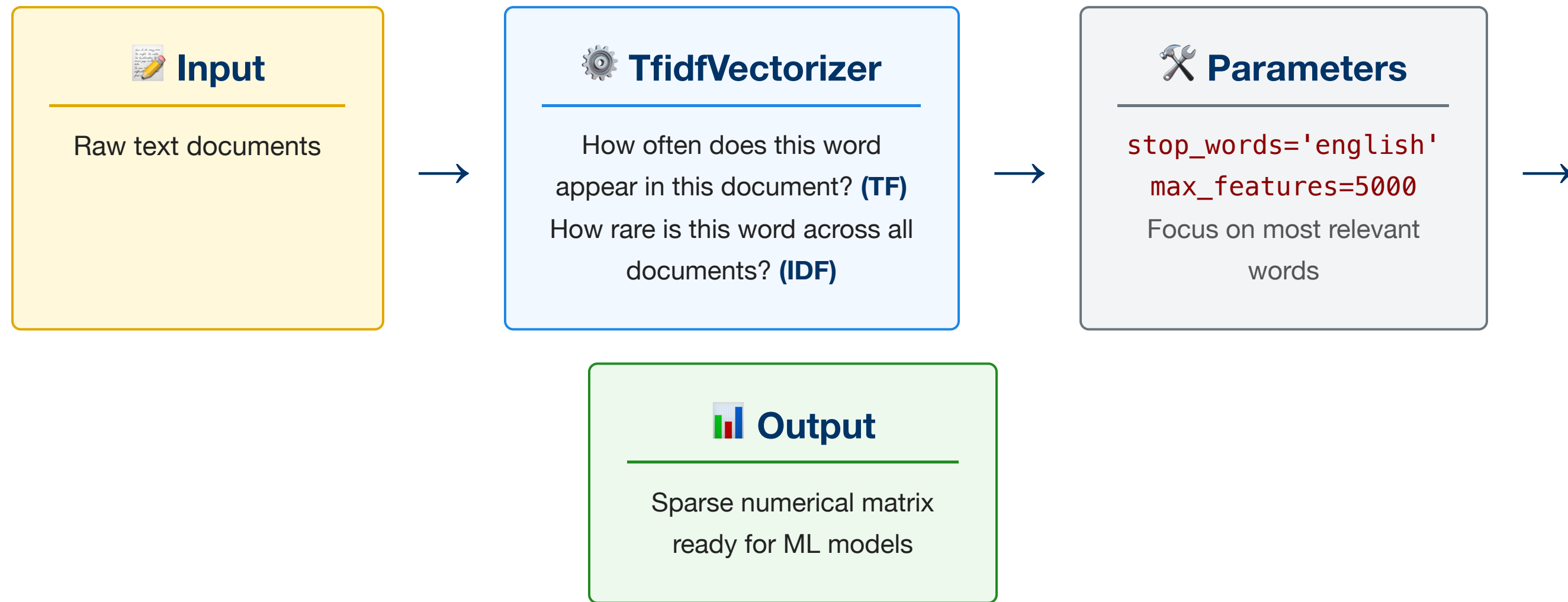
- **hotel** appears in D1 and D2, so it is less distinctive
- **clean** is frequent in D1 and rare overall, so its TF-IDF is high in D1
- **delay** dominates D3 and strongly signals a negative travel complaint

Tiny TF-IDF table

Term	DF	IDF intuition	Interpretation
hotel	2	Low	Common across hotel reviews, less informative.
clean	1	High	Strong signal for D1.
delay	1	High	Distinguishes D3 sharply.

TF-IDF does not understand meaning, but it often gives a surprisingly strong baseline.

How TF-IDF Vectorization Works



Part 1 takeaway

Classification means mapping text to labels. The main design choices are task type, feature representation, and model.

Part 2

Sentiment Analysis Deep Dive

What is sentiment analysis?

Polarity

positive, neutral, negative

Aspect-based

room = positive, service = negative

Emotion

joy, anger, sadness, surprise

Why businesses care

- Track customer satisfaction at scale
- Detect complaints early
- Compare competitors and products
- Support product teams with evidence from reviews
- Monitor reputation on social media

Key insight

A million reviews are useless if no one can summarize the emotional pattern inside them.

Rule-based vs. ML-based sentiment analysis

Approach	How it works	Best for	Limitation
Lexicon / rule-based	Matches words against sentiment dictionaries and heuristics.	Fast baselines, low-resource settings	Struggles with sarcasm and domain nuance
ML / transformer	Learns from labeled examples or pretraining.	Context-sensitive tasks	Needs compute, data, and validation

VADER in one slide

What VADER uses

- Sentiment lexicon with positive and negative scores
- Rules for capitalization, punctuation, negation, boosters
- Output scores: pos, neu, neg, compound

Why it is still useful

- No training needed
- Very interpretable
- Good classroom example of rule-based NLP

VADER demo logic

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
text = "The room was beautiful, but the staff were not helpful at all."
scores = analyzer.polarity_scores(text)
scores
```

Interpretation: the compound score summarizes overall polarity, but mixed reviews often hide aspect-level differences.

Zero-shot classification with transformers

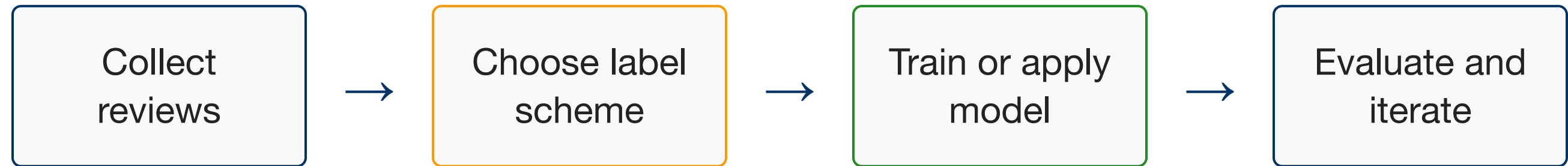
```
from transformers import pipeline
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
labels = ["positive", "neutral", "negative"]
classifier("The breakfast was fantastic but the room was noisy.", labels)
```

Zero-shot means we can classify new text without task-specific training, by framing labels as natural language hypotheses.

Live demo: 5 hotel reviews

Review	VADER	Transformer	Comment
"Fantastic breakfast and friendly staff."	Positive	Positive	Easy case
"The room was tiny, but the location was perfect."	Mixed	Mixed/neutral	Trade-off
"I waited 40 minutes to check in. Never again."	Negative	Negative	Strong complaint
"Not bad at all."	Positive	Positive	Negation rule matters
"Yeah, great service... if you enjoy being ignored."	Unclear	Negative	Sarcasm challenge

Sentiment analysis workflow



Pause slide

Break 

Come back in 10-15 minutes for evaluation metrics

Part 3

Evaluation Metrics

Why accuracy alone is not enough

Example

If 95% of emails are not spam, a model that always predicts "not spam" gets 95% accuracy and is still useless.

Lesson

We need metrics that reflect the kind of errors we actually care about.

Confusion matrix visual

Actual class	Predicted class	
	Positive	Negative
Positive	TP correct positive	FN missed positive
Negative	FP false alarm	TN correct negative

Precision, recall, F1

Precision

Of predicted positives, how many were truly positive?

$$\text{TP} / (\text{TP} + \text{FP})$$

Recall

Of actual positives, how many did we find?

$$\text{TP} / (\text{TP} + \text{FN})$$

F1-score

Balance precision and recall.

$$2\text{PR} / (\text{P} + \text{R})$$

Why Metrics Matter: A Business Case

Scenario

- An online hotel platform wants to predict customer churn
- Cost of losing a customer: **€100**
- Cost of sending a retention incentive: **€10**

Computed metrics

- Accuracy: **77%**
- Precision: **47%**
- Recall: **40%**

Actual outcome	Predicted churn	
	Yes	No
Churn	TP = 9	FN = 13
Stay	FP = 10	TN = 68

Total: 100 customers

The Cost Calculation

1 Do nothing

All churners leave

$$(9 + 13) \times \text{€}100 = \text{€}2,200 \text{ lost}$$

2 Use model (100% incentive success)

Send incentive to predicted churners

$$(19 \times \text{€}10) + (13 \times \text{€}100) = \text{€}1,490$$

Saves €710

3 Use model (only 10% success rate)

$$(19 \times \text{€}10) + (21 \times \text{€}100) = \text{€}2,290$$

Worse than nothing

Key Takeaway: Metrics & Business Context

Key Insight

High accuracy (77%) can hide poor recall (40%). In business contexts, **choosing the right metric depends on the cost structure**, not just model performance.

Rule of thumb: If the cost of missing a positive case is high (e.g., churn, disease) → optimize for **recall**. If the cost of a false alarm is high (e.g., fraud accusation, legal risk) → optimize for **precision**.

When to use which metric

Need high precision

- Legal risk alerts
- Fraud accusations
- Content moderation appeals

vs.

Need high recall

- Disease screening
- Spam filtering
- Complaint detection

Macro vs. micro vs. weighted

Average	How it aggregates	Good for	Risk
Macro	Treat each class equally, then average.	Fairness across classes	Sensitive to tiny classes
Micro	Pool all decisions first.	Overall system performance	Big classes dominate
Weighted	Average by class support.	Balanced reporting	Can hide minority failures

Pause & Practice

Compute the metrics from this confusion matrix

	Pred +	Pred -
Actual +	42	8
Actual -	18	132

Your task

1. Compute accuracy
2. Compute precision
3. Compute recall
4. Compute F1-score
5. Interpret the biggest weakness

Practice solution

Calculations

- Accuracy = $(42 + 132) / 200 = \mathbf{0.87}$
- Precision = $42 / (42 + 18) = \mathbf{0.70}$
- Recall = $42 / (42 + 8) = \mathbf{0.84}$
- F1 $\approx \mathbf{0.76}$

Interpretation

The model catches most positives, but 30% of positive predictions are wrong. That may be too many false alarms in some business settings.

ROC and AUC, briefly

ROC curve

Plots true positive rate against false positive rate across thresholds.

AUC

Single-number summary: higher means better class separation.

Placeholder visual: show ROC curve with diagonal baseline and model curve above it.

Class imbalance problem

What happens

- Rare classes get ignored
- Accuracy looks misleadingly high
- Threshold choices become critical

What we can do

- Use precision, recall, F1
- Resample or rebalance classes
- Adjust decision thresholds
- Collect more minority examples

Part 4

Classical ML vs. Transformers

Classical pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

clf = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("model", LogisticRegression(max_iter=1000))
])
```

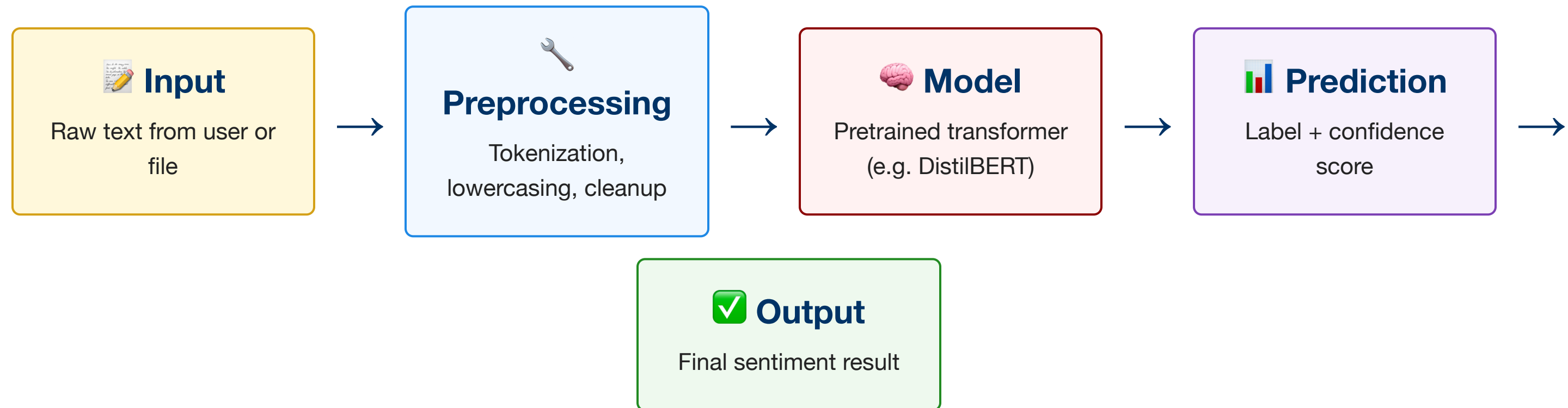
Simple, strong baseline. Easy to train, fast to run, and often surprisingly competitive on structured tasks.

Transformer pipeline

```
from transformers import pipeline
classifier = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)
classifier("The product is elegant and easy to use.")
```

Pretrained transformers capture context and semantics far better than sparse bag-of-words features.

Inside a Hugging Face Pipeline



Comparison table

Aspect	 Classical ML	 Transformers	Recommendation
Speed	Very fast	Slower	Classical for low-latency systems
Data needs	Works with less data	Benefits from more data	Start classical if labels are scarce
Interpretability	Higher	Lower	Classical for auditability
Context sensitivity	Limited	Strong	Transformers for nuance
Deployment cost	Low	Higher	Match model complexity to business value

Think-Pair-Share

When would you choose Naive Bayes over BERT?

Think

Individually note 2 conditions where a simpler model is preferable.

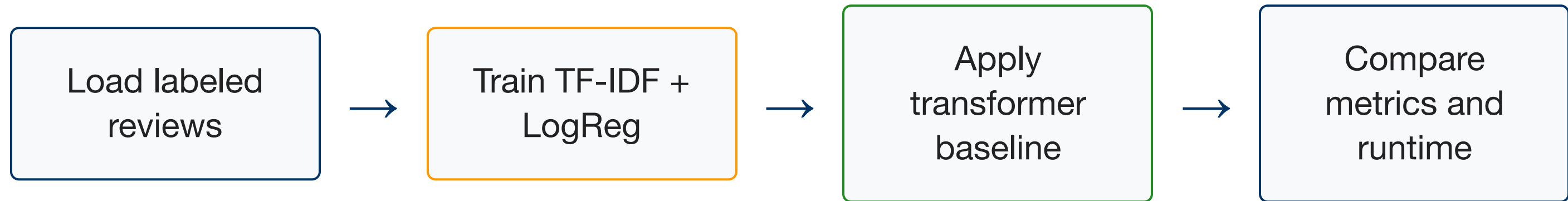
Pair

Compare your reasons with a neighbor.

Share

We collect examples on speed, data size, and interpretability.

Hands-on comparison workflow



Practical code sketch

```
# Classical baseline
clf.fit(train_texts, train_labels)
classic_preds = clf.predict(test_texts)

# Transformer inference
hf = pipeline("text-classification", model="distilbert-base-uncased-finetuned-sst-2-english")
transformer_preds = [hf(t)[0]["label"] for t in test_texts[:20]]
```

In real projects, make the comparison fair: same test set, same label mapping, same evaluation metrics.

Decision rules: when to use which

Choose classical ML if...

- You need speed and simplicity
- You have limited labeled data
- Interpretability matters a lot
- The task is stable and domain-specific

Choose transformers if...

- Context and nuance matter
- You face ambiguity or sarcasm
- You can afford more compute
- You want stronger zero-shot or transfer performance

Wrap-up

Main lessons

- Classification maps text to labels
- Sentiment analysis is one important classification task
- Metrics must match the error costs
- Classical and transformer models both have a place

Next module

Module 3 moves from classification pipelines to neural networks, embeddings, attention, and why transformers work so well.

Exit Ticket

Before you leave, answer these 3 questions

1 Concept

What is the difference between multi-class and multi-label classification?

2 Judgment

Which metric would you prioritize for complaint detection, and why?

3 Application

When is a classical model still the better business choice?

